Interpreting deep-learned error correcting codes

A. Mulgund, R. Shekhar, Y. Zhou
+ N. Devroye, G. Turán, M. Žefran

UNIVERSITY OF ILLINOIS CHICAGO

Error-correcting codes

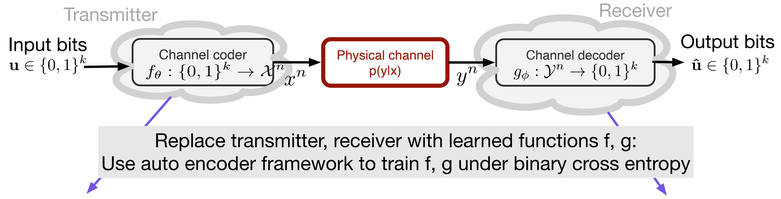
Error correcting codes (ECCs) are encoder / decoder pairs that add redundancy to bits so that when the codewords are corrupted, one can still recover them. Long, distinguished mathematical and algorithmic history / construction of ECCs includes convolutional codes, numerous algebraic constructions (e.g. BCH, Reed Solomon, Reed Muller, Hamming), Turbo codes, low-density parity-check (LDPC) codes, polar codes, many with capacity-approaching bit error rate (BER) performance.

For a given encoder, finding the optimal (ML, or MAP) decoder is in general exponential in the blocklength of the code; finding computationally reasonable codes with good BER or block error rate (BLER) performance is the goal.

Deep-learned error-correcting codes

Recently, deep learning has been used to learn the encoding and decoding functions from data instead of the manual derivation process, with remarkable robustness (to channel noise statistics) and performance over channels with feedback. We refer to such codes as deep-learned error-correcting codes (DL-ECC).

Our goal: develop tools to **interpret** the designed codes and training process. Interesting as interpretability problem and for coding theory.



First explored code: TurboAE [Jiang et al., NeurIPS 2019]

1. Train an Over Complete AutoEncoder
2. **Encoder** Neural Net $\mathcal{E}_\Theta$ with parameters Θ
3. Soft **Decoder** Neural Net $\mathcal{D}_\Phi$ with parameters Φ
4. **A**dditive **W**hite **G**aussian **N**oise (**AWGN**) channel between Encoder and decoder $AWGN_{\sigma^2} : x \mapsto x + z$ with $z \sim \mathcal{N}_{0,\sigma^2 I}$
5. **Training Objective**: Expected **B**inary **C**ross **E**ntropy (**BCE**)

$$\mathcal{L}(\Theta, \Phi) = \mathbb{E}_{\substack{x \sim \mathcal{U} \\ z \sim \mathcal{N}}} \left(\frac{1}{L} \sum_{i \in [L]} \text{BCE}(x_i, \mathcal{D}_\Phi(\mathcal{E}_\Theta(x) + z))_i \right)$$

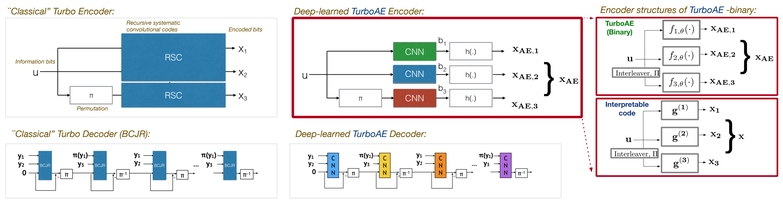


Figure 1. The rate $R = \frac{1}{3}$ ($\mathbf{u} \in \{0,1\}^{100}, \mathbf{x}_{AE,j} \in \{\pm 1\}^{100}$) classical Turbo and TurboAE encoder and decoder structures. The recursive systematic (RSC) convolutional codes and the BCJR decoding blocks are both replaced by convolutional neural network (CNN) blocks. Functions $f_{j,\theta}(\cdot)$ are the constituent codes implemented as CNNs. Our interpretable codes either provide compact Boolean representations for the learned functions $f_{j,\theta}$ or approximate them with modified convolutional codes $\mathbf{g}^{(j)}$.

Approximating the TurboAE-encoder

1. Estimate the **influence** (prob that if you change one input bit you change the output bit) of input bits on output bits of encoder function to determine how many input bits affect output bits.
2. In TurboAE-binary 5 consecutive input bits affect each output (memory 5); TurboAE-cont memory 9.
3. Find "closest" linear approximation to constituent encoding functions using mixed integer linear programming. Alternatively, Goldreich Levin is an efficient algorithm that can estimate the largest FCs of the (black box) encoder – useful general tool.
4. Use influence analysis to find exact non-linear encoding functions (lucky).
5. Compare performance of these approximations + interpretable BCJR decoder.

Block #	Approximate expressions for output #i
1	$1 \oplus u_1 \oplus u_2 \oplus u_3 \oplus u_4 \oplus u_5$
2	$u_1 \oplus u_3 \oplus u_4 \oplus u_5$
3	Solution 1: $u_1 \oplus u_2 \oplus u_4$ Solution 2: $1 \oplus u_1 \oplus u_2 \oplus u_3 \oplus u_4$ Solution 3: $1 \oplus u_1 \oplus u_2 \oplus u_4 \oplus u_5$ Solution 4: $1 \oplus u_1 \oplus u_2 \oplus u_3 \oplus u_4 \oplus u_5$
where	$u_1 = x_{i+2}$, $u_2 = x_{i+1}$, $u_3 = x_j$, $u_4 = x_{i-1}$, $u_5 = x_{i-2}$, $x =$ inputs

TABLE I: Best affine approximations for TurboAE's encoder. Block 3 has four equally good approximations.

Block #	Expression for output #$i \in \{3, 4, \cdots, 98\}$
1	$1 \oplus u_1 \oplus \bar{u}_2 \oplus u_3 \oplus \bar{u}_4 \oplus u_5$ $\oplus\ \bar{u}_2 u_3 \bar{u}_4 \oplus u_1 \bar{u}_2 u_3 \bar{u}_4 u_5$
2	$u_1 \oplus u_3 \oplus u_4 \oplus u_5$
3	$u_1 \oplus u_2 \oplus u_4 \oplus \bar{u}_3 \bar{u}_5$
where	$u_1 = x_{i+2}$, $u_2 = x_{i+1}$, $u_3 = x_i$, $u_4 = x_{i-1}$, $u_5 = x_{i-2}$, $x =$ inputs

TABLE II: Exact expressions for TurboAE Encoder's non-boundary bits.

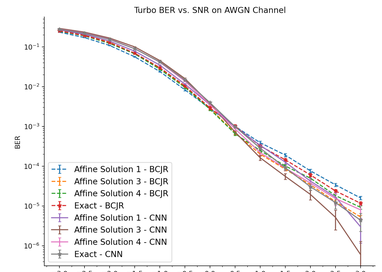


Fig. 4: Comparison of TurboAE's nonlinear encoder with its affine approximations. Solution 3 is the best-performing approximation and solution 1 the worst. Each is paired with either a BCJR decoder (dashed line) or a CNN decoder (solid line). Exact encoders' BER points are marked with stars. Solution 1 consistently underperforms TurboAE, while Solution 3 consistently outperforms it. CNN decoder consistently outperforms the BCJR decoder above SNR 0.5 when paired with the same encoder.

Figure 2. Comparison of TurboAE's encoder with its affine approximations. Solution 3 is the best and outperforms TurboAE. Solution 1 is the worst. Each is paired with either a BCJR (dashed line) or a CNN decoder (solid). CNN outperforms BCJR decoder above SNR 0.5 given same encoder.

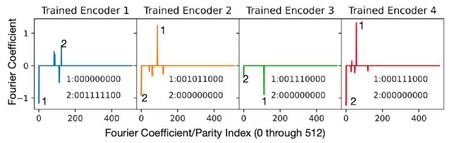
A Fourier lens

Fourier representation of a Boolean function $f : \{-1,1\}^n \longrightarrow \{-1,1\}$

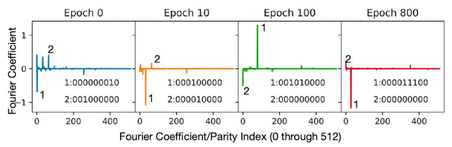
$$f(x) = \sum_{S \subseteq \{1,2,\cdots,n\}} \hat{f}(S) \chi_S(x),$$

where $\hat{f}(S)$ is the **F**ourier **C**oefficient **(FC)** for set S, and $\chi_S(x) = \prod_{i \in S} x_i$.

Can look at how FCs evolve over training. At initialization, the dominant FCs almost always correspond to one bit parities. However, with training, higher degree parities emerge as dominant (see figure). Although the setups differ, these observations may be related to recently observed staircase properties [Abbe et al., NeurIPS 2021].



(a) Largest FCs of Block 1 with energy $\geq 95\%$ over 4 independent training sessions. Always a few FCs dominate the Fourier space.



(b) FCs of block 3 over different epochs of a training session. Initially, 1-bit parities always dominate, but later higher degree parities emerge.

Training and the BCE

Consider $U \sim \text{Unif}[\mathbb{F}_2^k]$ to be our input message, $Y^{(\Theta)}$ to be the corrupted codeword, parameterized by Θ, and $V^{(\Theta,\Phi)}$ to be our soft decoding, with decoder parameters Φ. We can decompose the expected BCE on bit $i \in [k]$ as

$$\mathcal{L}(\Theta, \Phi) := \mathbb{E}\left[\text{BCE}(U_i, V_i^{(\Theta,\Phi)})\right] = D_{KL}\left(\mathbb{P}[U_i|Y^{(\Theta)}] \| V_i^{(\Theta,\Phi)}|Y^{(\Theta)}\right) + \mathbb{H}(U_i|Y^{(\Theta)})$$

$\mathbb{H}(U_i|Y)$ depends only on the *encoder* and $D_{KL}\left(\mathbb{P}[U_i|Y] \| V_i|Y\right)$ is minimized to 0 precisely when $V_i = \mathbb{P}[U_i|Y]$ (i.e. when Φ parameterizes the *MAP decoder*).

Junction Tree Algorithm and its insight into training

Because the TurboAE encoder is a CNN, it defines a Turbo Code made up of non-systematic convolutional codes of window 9. This means we can factor

$$\mathbb{P}[U_i = u|Y] \propto \sum_{U \in \mathbb{F}_2^k, U_i = u} \prod_{j=1}^{k} \mathbb{P}[Y_j | U_{\texttt{wind}}]$$

where `wind` is the set of bits that Y_j depends on. This defines a factor graph, and we can apply the Junction Tree algorithm to compute $\mathbb{P}[U_i|Y]$ for all $i \in [k]$. We then use $\mathbb{P}[U_i|Y]$ to estimate the decomposition terms of BCE during training.

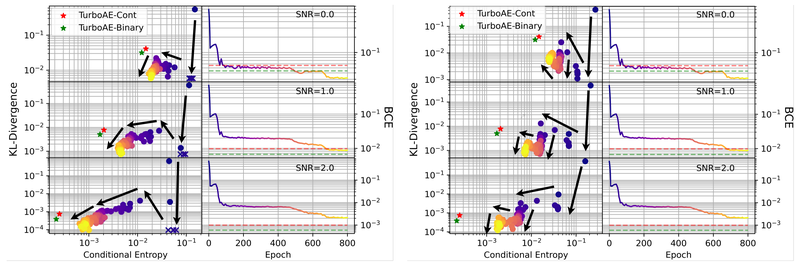


Figure 3. Decomposition evolution of two training runs of TurboAE for $k = 40$. Each dot is 8 epochs. Purple corresponds to earlier epochs and yellow to later epochs. Note the zig-zag pattern in the training. The optimization process escapes a plateau by improvements in the encoder. The decompositions for both variants of TurboAE with decoder retrained for $k = 40$ are plotted as well.

These results bring up some new questions:

1. What properties determine how long it takes to improve the encoder?
2. Do we see similar zig-zag patterns when training other DL-ECCs?

References

[1] E. Abbe, E. Boix-Adsera, M. S. Brennan, G. Bresler, and D. Nagaraj.
The staircase property: How hierarchical structure can guide deep learning.
In *Advances in Neural Information Processing Systems*, volume 34, pages 26989–27002, 2021.

[2] N. Devroye, N. Mohammadi, A. Mulgund, H. Naik, R. Shekhar, Gy. Turán, Y. Wei, and M. Žefran.
Interpreting deep-learned error-correcting codes.
In *2022 IEEE International Symposium on Information Theory (ISIT)*, pages 2457–2462, 2022.

[3] N. Devroye, A. Mulgund, R. Shekhar, Gy. Turán, M. Žefran, and Y. Zhou.
Interpreting training aspects of deep-learned error-correcting codes.
In *2023 IEEE International Symposium on Information Theory (ISIT)*, pages 2374–2379, 2023.

[4] Y. Jiang, H. Kim, H. Asnani, S. Kannan, S. Oh, and P. Viswanath.
Turbo autoencoder: Deep learning based channel codes for point-to-point communication channels.
In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[5] A. Mulgund, R. Shekhar, N. Devroye, Gy. Turán, and M. Žefran.
Evaluating interpretations of deep-learned error-correcting codes.
In *2022 58th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, pages 1–8, 2022.

NSF Tripods 1934915; NSF IDEAL 2217023

mulgund2@uic.edu, rshekh3@uic.edu, yzhou238@uic.edu